DSA — SORTING ALGORITHMS

Integrated supplement for your combined DSA notes

This section is based on the uploaded 19-page lecture note titled “Sorting Algorithms”. It covers the six algorithms named in that source and its lower-bound analysis. The source emphasizes incremental design, divide and conquer, traces, correctness invariants, and running-time analysis.

> Exam answer pattern: define the algorithm → explain its idea → write pseudocode → trace a small input → state best/worst time and extra space → mention the key property.

1. Sorting: the big picture

Sorting arranges items in ascending or descending order. The uploaded lecture note covers Bubble Sort, Insertion Sort, Selection Sort, Merge Sort, Quick Sort and Heap Sort.

Algorithm	Design idea	Best time	Worst time	Key point
Bubble Sort	Adjacent comparisons and exchanges	$\Theta(n)$ only with early-stop optimization; source's fixed-pass version still compares $\Theta(n^2)$	$\Theta(n^2)$	Largest remaining item moves to the end each pass.
Insertion Sort	Incremental design	$\Theta(n)$	$\Theta(n^2)$	Insert each next item into the sorted prefix.
Selection Sort	Repeatedly select minimum	$\Theta(n^2)$	$\Theta(n^2)$	Find minimum in remaining suffix and swap into place.
Merge Sort	Divide and conquer	$\Theta(n \log n)$	$\Theta(n \log n)$	Split, recursively sort, then merge.
Quick Sort	Divide and conquer + partition	$O(n \log n)$ expected/balanced	$\Theta(n^2)$	Pivot partitions items into smaller/larger groups.
Heap Sort	Heap data structure	$\Theta(n \log n)$	$\Theta(n \log n)$	Build max-heap; repeatedly move maximum to end.

The lecture notes' concluding overview states that Bubble, Selection, Insertion and Quick Sort have quadratic worst-case behaviour, while Merge Sort and Heap Sort have O(n log n) worst-case behaviour. ■filecite■turn10file0■L758-L760■

2. Bubble Sort

Bubble Sort is comparison-based. It compares adjacent elements and swaps them when they are out of order. After each pass, the largest item still unsorted reaches its correct position at the right end.

Pseudocode (ascending order)

```
BUBBLE_SORT(A, n)
    for i ← 1 to n - 1
        for j ← 1 to n - i
            if A[j] > A[j + 1]
                swap(A[j], A[j + 1])
```

Invariant and complexity

Invariant: after pass i, the last i positions contain the i largest elements in sorted order. The source's pseudocode performs n – i comparisons on pass i, so total comparisons are (n–1)+(n–2)+…+1 = n(n–1)/2 = $\Theta(n^2)$, regardless of input order. If no swaps occur in an already-sorted input, an optimized early-stop version can finish in $\Theta(n)$; that optimization is not in the source pseudocode.

Trace

Pass	Array after pass (input: 5, 1, 4, 2)	What is fixed?
Start	[5, 1, 4, 2]	—
1	[1, 4, 2, 5]	5 is at the end.
2	[1, 2, 4, 5]	4 is in its final position.
3	[1, 2, 4, 5]	Sorted.

> Bubble Sort memory cue: adjacent swap; largest bubbles right. Stable when equal elements are not swapped.

3. Insertion Sort

Insertion Sort follows incremental design: the first item is considered sorted, then each next key is inserted into its correct position within the already-sorted prefix.

```
INSERTION_SORT(A, n)
    for j ← 2 to n
        key ← A[j]
        i ← j - 1
        while i > 0 and A[i] > key
            A[i + 1] ← A[i]
            i ← i - 1
        A[i + 1] ← key
```

Trace

Step	Action	Array
Initial	—	[-1, 4, 7, 2, 3]
Insert 4	Already in correct place	[-1, 4, 7, 2, 3]
Insert 7	Already in correct place	[-1, 4, 7, 2, 3]
Insert 2	Shift 7 and 4 right; place 2	[-1, 2, 4, 7, 3]
Insert 3	Shift 7 and 4 right; place 3	[-1, 2, 3, 4, 7]

Complexity

Best case: ascending input; each key needs only one comparison with the previous item, giving $\Theta(n)$. Worst case: descending input; each key may move across the whole sorted prefix, giving $\Theta(n^2)$. It is in-place and stable in the usual implementation.

> Memory cue: take the next key, shift larger values right, and insert the key into the gap.

4. Selection Sort

Selection Sort repeatedly finds the minimum element in the unsorted suffix and swaps it with the first position of that suffix. After iteration i, position i contains the correct i-th smallest item.

```
SELECTION_SORT(A, n)
    for j ← 1 to n - 1
        min ← j
        for i ← j + 1 to n
            if A[i] < A[min]
                min ← i
        swap(A[j], A[min])
```

Trace

Pass	Minimum in remaining suffix	Array after swap
Start	—	[4, 2, 5, 1]
1	1	[1, 2, 5, 4]
2	2	[1, 2, 5, 4]
3	4	[1, 2, 4, 5]

Complexity

The inner comparison executes n–1, n–2, …, 1 times. Therefore the comparison count is $\Theta(n^2)$ in both best and worst cases. Selection Sort generally performs only O(n) swaps, but its comparisons remain quadratic.

> Memory cue: select the minimum; put it in the next final position. Unlike insertion sort, already-sorted input does not remove the quadratic comparisons.

5. Merge Sort

Merge Sort uses divide and conquer. Divide the array into halves until subarrays contain one element; recursively sort the halves; merge the sorted halves into one sorted array.

```
MERGE_SORT(A, p, r)
    if p < r
        q ← floor((p + r) / 2)
        MERGE_SORT(A, p, q)
        MERGE_SORT(A, q + 1, r)
        MERGE(A, p, q, r)
```

Merge routine — core idea

Use two pointers, one for each sorted half. Repeatedly copy the smaller front element to the output, advance that pointer, then copy any remaining elements when one half is exhausted.

Split and combine example

Stage	Result
Input	[1, 4, 2, 8, -1, 0]
Split	[1, 4, 2] and [8, -1, 0]
Sort left half	[1, 2, 4]
Sort right half	[-1, 0, 8]
Merge	[-1, 0, 1, 2, 4, 8]

Complexity

Recurrence: $T(n) = 2T(n/2) + \Theta(n)$, because two half-size subproblems are solved and merging takes linear time. The result is $\Theta(n \log n)$ in best, average and worst cases. Standard array Merge Sort requires $\Theta(n)$ auxiliary space.

Memory cue: split → sort halves → merge. The source's split-phase and combine-phase diagrams are on page 8; the recursive trace and stack explanation appear on pages 9–10.

6. Quick Sort

Quick Sort uses divide and conquer. Choose a pivot, partition the array so elements ≤ pivot come before it and larger elements after it, then recursively sort the two sides. The uploaded source uses the last element as pivot.

```
QUICK_SORT(A, p, r)
    if p < r
        q ← PARTITION(A, p, r)
        QUICK_SORT(A, p, q - 1)
        QUICK_SORT(A, q + 1, r)

PARTITION(A, p, r)
    pivot ← A[r]
    i ← p - 1
    for j ← p to r - 1
        if A[j] ≤ pivot
            i ← i + 1
            swap(A[i], A[j])
    swap(A[i + 1], A[r])
    return i + 1
```

Partition trace

Step	Array	Meaning
Start	[2, 8, 7, 1, 3, 5]	Pivot = 5.
Compare 2	[2, 8, 7, 1, 3, 5]	2 ≤ 5; keep in left region.
Compare 8, 7	[2, 8, 7, 1, 3, 5]	Both > 5; stay in right region.
Compare 1	[2, 1, 7, 8, 3, 5]	Swap 1 into left region.
Compare 3	[2, 1, 3, 8, 7, 5]	Swap 3 into left region.
Place pivot	[2, 1, 3, 5, 7, 8]	Pivot 5 reaches its final position.

Complexity

Partition takes $\Theta(n)$. Balanced or near-balanced partitions give $\Theta(n \log n)$. If each pivot leaves one subproblem of size $n-1$ and one empty/small side, the recurrence is $T(n)=T(n-1)+\Theta(n)$, yielding $\Theta(n^2)$. Choosing the last item as pivot can cause this worst case on already increasing or decreasing arrays.

> Memory cue: pivot → partition → recursively sort left and right. Quick Sort is often fast in practice, but its worst case is quadratic with poor pivot splits.

7. Heap Sort

A heap is a nearly complete binary tree. In a max-heap, every parent is at least as large as its children, so the maximum element is at the root. With 1-based array indexing, the left child of i is 2i and the right child is 2i+1.

Heap Sort process

1. Build a max-heap. 2. Swap the root (maximum) with the last item in the heap. 3. Reduce the heap size by one. 4. Max-heapify from the root. 5. Repeat until one item remains.

```
HEAP_SORT(A)
    BUILD_MAX_HEAP(A)
    for i ← length(A) down to 2
        swap(A[1], A[i])
        heap_size ← heap_size - 1
        MAX_HEAPIFY(A, 1)
```

Max-heapify

Compare the node with its left and right children, identify the largest, swap if a child is larger, then continue heapifying down the affected subtree.

Complexity

Task	Complexity
Build heap using bottom-up method	Θ(n)
One max-heapify operation	O(log n)
Repeated extraction / sorting phase	Θ(n log n)
Heap Sort overall	Θ(n log n) worst case
Extra space	O(1) for standard in-place array Heap Sort, excluding implementation details

The lecture source distinguishes top-down heap construction (O(n log n)) from bottom-up construction (O(n)); it then gives Heap Sort worst-case time O(n log n). ■filecite■turn10file0■L684-L690■ ■filecite■turn10file0■L730-L744■

> Memory cue: max-heap keeps the largest at the root; move it to the end, shrink heap, heapify again.

8. Lower-bound analysis

Lower-bound analysis studies the minimum amount of work required by any algorithm for a problem, rather than analyzing only one specific algorithm.

Search lower bounds from the source

Problem	Lower bound	Reasoning
Search in an arbitrary array	Ω(n)	In the worst case, every item may need to be checked.
Search in a sorted array	Ω(log n)	Comparison decisions can eliminate only portions of the search space per level.
Comparison-based sorting	Ω(n log n)	A decision tree must distinguish n! possible input permutations.

Why comparison sorting needs Ω(n log n)

A comparison decision tree has a leaf for each possible ordering. There are n! possible permutations, so the tree needs at least n! leaves. A binary tree of height h has at most 2■ leaves, hence 2■ ≥ n!, so h ≥ log■(n!). Using the standard estimate for log(n!), this gives Ω(n log n).

The source concludes that Merge Sort and Heap Sort match the comparison-sorting lower bound in worst-case asymptotic order and are therefore optimal with respect to number of comparisons. ■filecite■turn10file0■L814-L831■

> Important qualification: this lower bound is for comparison-based sorting. The source explicitly notes that counting sort and radix sort do not fit that comparison model and can run in O(n) under suitable assumptions. ■filecite■turn10file0■L816-L821■

9. Final sorting comparison sheet

Algorithm	Best	Average (typical)	Worst	Stable?	In-place? (typical)
Bubble	Θ(n) if early-stop; otherwise Θ(n²)	Θ(n²)	Θ(n²)	Yes	Yes
Insertion	Θ(n)	Θ(n²)	Θ(n²)	Yes	Yes
Selection	Θ(n²)	Θ(n²)	Θ(n²)	Usually no	Yes
Merge	Θ(n log n)	Θ(n log n)	Θ(n log n)	Yes	No for standard array version
Quick	Θ(n log n)	Expected Θ(n log n)	Θ(n²)	Usually no	Usually yes (in-place partition; recursion stack remains)
Heap	Θ(n log n)	Θ(n log n)	Θ(n log n)	No	Yes

Stability and in-place properties are standard properties of common implementations; the uploaded lecture primarily emphasizes algorithmic design, traces, invariants and running-time analysis.

Quick exam prompts

1. Trace Bubble Sort and state its pass invariant.
2. Explain why Insertion Sort is Θ(n) on ascending input and Θ(n²) on descending input.
3. Show why Selection Sort always performs Θ(n²) comparisons.
4. Explain Merge Sort using divide and conquer and solve T(n)=2T(n/2)+Θ(n).
5. Trace Quick Sort partition when the last item is the pivot; discuss its worst case.
6. Explain max-heap, max-heapify, bottom-up heap construction and Heap Sort.
7. Prove that comparison-based sorting has lower bound Ω(n log n).

End of sorting supplement. Use this chapter together with the earlier DSA notes on arrays, linked lists, stacks, queues and algorithm analysis.